Bapuji Educational Association (Regd.)

Bapuji Institute of Engineering and Technology

An Autonomous Institute Affiliated to Visvesvaraya Technological University, Belagavi, Karnataka.

Department of Artificial Intelligence & Machine Learning

Analysis and Design of Algorithms (ADA)

Module 4 – The Greedy Method

The Greedy Method

- Prim's Algorithm
- Kruskal's Algorithm
- Dijkstra's Algorithm
- Huffman Trees and Codes

Prepared by:

Dr. Naveen Kumar K R

(Exam Revision Notes)

The Greedy Method: Prim’s Algorithm

1. What is Prim’s Algorithm?

- **Definition:** Prim’s algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a weighted, connected, and undirected graph.
- **Purpose:** It finds a subset of edges that connects all vertices in the graph without any cycles, while keeping the total weight of the edges as small as possible.
- **Greedy Strategy:** At every step, the algorithm adds the "nearest" vertex to the existing tree. This means it always picks the cheapest edge that connects a vertex already in the tree to one that is not yet in the tree.
- **Time Complexity:**
  - $O(|V|^2)$: When using a simple weight matrix or an unordered array for the priority queue.
  - $O(|E| \log |V|)$: When using an adjacency list and a **min-heap** (priority queue).

2. Key Idea Behind Prim’s Algorithm

1. **Start Small:** Choose any arbitrary vertex to be the root of your tree (in this problem, we start at **a**).
2. **Maintain Sets:** Keep track of **Tree Vertices** (visited) and **Non-Tree Vertices** (the "fringe" or "unseen").
3. **Choose the Best Edge:** Look at all edges connecting the current tree to vertices outside the tree. Select the edge with the **minimum weight**.
4. **Grow the Tree:** Add that minimum edge and the new vertex to your tree.
5. **Repeat:** Continue this process until every vertex in the graph is part of the tree.

3. Prim’s Algorithm – Pseudocode

Here is the standard logical structure for Prim's algorithm using keys and parent pointers:

```
Algorithm Prim(G, w, r) // G = Graph, w = weights, r = start vertex
for each u in V do
    key[u] ← ∞        // key[u] stores the minimum weight to connect u to the tree
    parent[u] ← null  // parent[u] stores the tree vertex that u connects to
key[r] ← 0            // Starting vertex has 0 weight to connect to itself
Q ← all vertices in V // Priority Queue containing all vertices based on key
while Q is not empty do
    u ← Extract-Min(Q) // Pick vertex u with the smallest key
    for each v adjacent to u do
        if v is in Q and w(u,v) < key[v] then
            parent[v] ← u
            key[v] ← w(u,v) // Update the best way to reach vertex v
```

Explanation of terms:

- `key[v]`: The minimum weight edge connecting vertex v to any vertex already in the tree.
- `parent[v]`: The vertex already in the tree that provides this minimum weight connection.
- `Extract-Min`: Removing the vertex with the lowest weight from the priority queue.

4. Applying Prim's Algorithm to the Given Graph

Graph Representation:

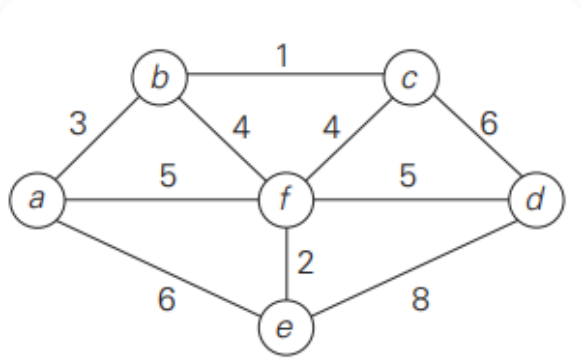


Figure: Weighted undirected graph used in the trace (vertices a,b,c,d,e,f).

- **Vertices:** {a, b, c, d, e, f}
- **Edges (E):** (a,b,3), (b,c,1), (c,d,6), (a,f,5), (b,f,4), (c,f,4), (f,d,5), (f,e,2), (a,e,6), (e,d,8)

Step-by-Step Trace Table (Starting from vertex 'a')

Step	Vertex Added (u)	Tree Vertices (VT)	Fringe Edges (u,v) with weights	Updated Keys / Priority Queue	Edge Selected	Weight	Updated MST Edges
0	-	{}	-	$key[a] = 0$, others ∞	-	-	{}
1	a	{a}	(a,b):3, (a,f):5, (a,e):6	$b:3, f:5, e:6, c:\infty, d:\infty$	(a,b)	3	{(a,b)}
2	b	{a, b}	(b,c):1, (b,f):4	$c:1, f:4, e:6, d:\infty$	(b,c)	1	{(a,b), (b,c)}
3	c	{a, b, c}	(c,d):6, (c,f):4	$f:4, d:6, e:6$	(b,f)	4	{(a,b), (b,c), (b,f)}
4	f	{a, b, c, f}	(f,d):5, (f,e):2	$e:2, d:5$	(f,e)	2	{(a,b), (b,c), (b,f), (f,e)}
5	e	{a, b, c, f, e}	(e,d):8	$d:5$ (since 5 < 8)	(f,d)	5	{(a,b), (b,c), (b,f), (f,e), (f,d)}
6	d	{a, b, c, f, e, d}	-	Done	-	-	Final MST

Note: In Step 3, both (b,f) and (c,f) have a weight of 4. We chose (b,f) arbitrarily.

5. Final Minimum Spanning Tree and Total Cost

- **MST Edges:** (a,b), (b,c), (b,f), (f,e), (f,d)
- **Weight Calculation:** $3 + 1 + 4 + 2 + 5 = 15$

The minimum spanning tree consists of edges: {(a,b), (b,c), (b,f), (f,e), (f,d)}. Total cost = 15.

6. Examination Answer Writing Format (For 8 Marks)

1. **Definition & Strategy (1 Mark):** Briefly define Prim's as a greedy algorithm for finding MST.
2. **Pseudocode (2 Marks):** Write the key-based pseudocode neatly. Label steps clearly.
3. **Initialization (1 Mark):** Show the initial state where $key[start] = 0$ and others are ∞.
4. **Trace Table (3 Marks):** Present the step-by-step construction in a table like the one above. This is where you earn the most marks.
5. **Final Result (1 Mark):** List the final edges and state the total cost clearly.

7. Common Mistakes to Avoid

- **Starting weight:** Forgetting to set the starting vertex's key to 0; it should be the first one extracted.
- **Updating keys:** Only update a neighbor's key if the new edge weight is **smaller** than its current key value.
- **Queue Membership:** Only update vertices that are still in the Priority Queue (Q). If a vertex is already in the Tree (VT), ignore it to avoid cycles.
- **Prim vs. Kruskal:** Remember that Prim builds **one single tree** that grows from a root, while Kruskal's builds a forest of separate edges that eventually join.
- **Arithmetic:** Double-check the final addition of edge weights!

8. Quick Revision

- **Strategy:** Greedy (nearest neighbor).
- **Key Property:** Key stores the min weight to the tree, parent stores the connection.
- **Time:** $O(V^2)$ or $O(E \log V)$.
- **For this graph:** MST Cost = **15**. Edges = **(a,b), (b,c), (b,f), (f,e), (f,d)**.

The Greedy Method: Kruskal's Algorithm for MST

1. What is Kruskal's Algorithm?

- **Definition:** Kruskal's algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a weighted, connected, undirected graph.
- **Purpose:** It finds a subset of edges that connects all vertices with the minimum possible total edge weight, ensuring no cycles are formed.
- **Greedy Nature:** It processes edges by always choosing the next "cheapest" (lowest weight) edge available, regardless of where it is in the graph.
- **High-level Idea:**
  1. Start with a "forest" where every vertex is its own separate tree.
  2. Sort all edges in non-decreasing order of weight.
  3. For each edge, if it connects two different trees, add it to the MST and merge the trees.
- **Time Complexity:** $O(|E| \log |E|)$, which is dominated by the time needed to sort the edges.

2. Why Must Cycles Be Avoided?

- **Tree Definition:** A spanning tree with V vertices must have exactly $V - 1$ **edges** and **no cycles**.
- **Redundancy:** Adding an edge that forms a cycle means you are connecting two vertices that are already connected by another path.
- **Optimality:** A cycle-forming edge adds extra weight to the total cost without providing any new connectivity. To keep the weight "minimum," we must reject such edges.
- **Algorithm Rule:** Kruskal's algorithm only accepts an edge if it connects two previously disconnected components.

3. Disjoint Subsets (Union-Find) for Cycle Detection

To efficiently check if adding an edge (u, v) creates a cycle, we use the **Disjoint Set Union (DSU)** or **Union-Find** data structure.

The Abstract Data Type (ADT) Operations:

1. `Makeset(v)`: Creates a one-element set containing only vertex v. Initially, every vertex is its own representative.
2. `Find(v)`: Returns the "representative" (or root) of the set containing v. If `Find(u) == Find(v)`, then u and v are already in the same component, and adding an edge between them will create a cycle.
3. `Union(u, v)`: Merges the two sets containing u and v into a single set.

Efficiency Heuristics:

- **Union by Rank/Size**: To keep the trees shallow, always attach the smaller tree under the root of the larger tree.
- **Path Compression**: During a `Find` operation, make every node on the path point directly to the root, flattening the structure for future operations.
- **Result**: These heuristics make Union-Find operations nearly constant time, meaning the sorting of edges becomes the main time-consuming part of Kruskal's algorithm.

4. Illustrative Trace (Using a Small Graph)

We will trace the algorithm using the graph from the sources (Vertices: {a, b, c, d, e, f}).

Step 1: Sort all edges by weight

Edge	(b,c)	(e,f)	(a,b)	(b,f)	(c,f)	(a,f)	(d,f)	(a,e)	(c,d)	(d,e)
Weight	1	2	3	4	4	5	5	6	6	8

Step 2: Initialize Sets (Makeset)
Initial sets: `{a}, {b}, {c}, {d}, {e}, {f}`.

Step 3: Process Edges in Order

Edge (u,v)	Find(u)	Find(v)	Action	Reason	Updated Sets / Components
(b,c)	b	c	Accept	Different sets	`{a}, {b,c}, {d}, {e}, {f}`
(e,f)	e	f	Accept	Different sets	`{a}, {b,c}, {d}, {e,f}`
(a,b)	a	b	Accept	Different sets	`{a,b,c}, {d}, {e,f}`
(b,f)	b	e	Accept	Different sets	`{a,b,c,e,f}, {d}`
(c,f)	b	b	Reject	Cycle!	Both already in `{a,b,c,e,f}`
(a,f)	b	b	Reject	Cycle!	Both already in `{a,b,c,e,f}`
(d,f)	d	b	Accept	Different sets	`{a,b,c,d,e,f}`

Note: Once all vertices are in one set, the algorithm stops.

Step 4: Final Result

- **MST Edges:** (b,c), (e,f), (a,b), (b,f), (d,f).
- **Total Weight:** $1 + 2 + 3 + 4 + 5 = 15$.

5. Visual Representation of Union-Find

- **Before Union(b,f):** We have two trees. Tree 1 has root 'b' (containing a, b, c). Tree 2 has root 'e' (containing e, f).
- **During Union(b,f):** We point the root of Tree 2 ('e') to the root of Tree 1 ('b').
- **Path Compression:** If we later call `Find(f)`, the algorithm will update 'f' to point directly to 'b', skipping 'e' for faster future searches.

6. Examination Answer Writing Format (8 Marks)

1. **Kruskal's Definition (1 Mark):** Briefly define it as a greedy algorithm for MST that sorts edges first.
2. **Pseudocode (2 Marks):** Provide the sorted-edge-based while loop.

3. **Cycle & Union-Find Explanation (2 Marks):** Explain that `Find` checks if vertices are in the same component and `Union` merges them to avoid cycles.
4. **Trace Table (2 Marks):** Draw a table showing Edge, Weight, and Accept/Reject status.
5. **MST & Total Cost (1 Mark):** State the final edges and the sum of weights.

7. Common Mistakes

- **Not Sorting:** Jumping to edges without sorting them by weight first. Sorting is the most important step.
- **Prim vs. Kruskal:** Prim builds one tree from a single starting vertex; Kruskal builds many small trees (forest) that eventually merge.
- **Misidentifying Cycles:** Attempting to use DFS/BFS for every edge. This is too slow; always mention **Union-Find** for cycle detection.
- **Edge Count:** Forgetting that an MST must have exactly $V - 1$ **edges**.

8. Quick Revision

- **Kruskal:** Greedy, sorts edges, builds forest.
- **Union-Find:** `Find` for cycle check, `Union` to join components.
- **MST Properties:** Connected, no cycles, $V - 1$ edges.
- **Complexity:** $O(E \log E)$ due to sorting.
- **Efficiency:** Use **Path Compression** and **Union by Rank** for $O(1)$ amortized DSU operations.

The Greedy Method: Kruskal's Algorithm

1. What is Kruskal's Algorithm?

- **Definition:** Kruskal's algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a connected, weighted, and undirected graph.
- **Purpose:** It aims to find a subset of edges that connects all vertices in the graph without any cycles, while minimizing the total sum of edge weights.
- **Greedy Strategy:** The algorithm processes all edges in increasing order of their weights; it adds an edge to the MST if it connects two previously unconnected components (does not form a cycle).
- **Time Complexity:** The complexity is $O(|E| \log |E|)$, primarily dominated by the time needed to sort the edges.
- **Contrast with Prim's Algorithm:** While Prim's algorithm grows a single tree from a starting vertex, Kruskal's algorithm starts with a "forest" of separate vertices and merges them into a single tree by picking the smallest available edges from anywhere in the graph.

2. Kruskal's Algorithm – Pseudocode

The following pseudocode outlines the standard logic using Union-Find operations for cycle detection:

```
Algorithm Kruskal(G, w)
// G = (V,E), w = weights
A ← ∅
for each vertex v in V do
    Make-Set(v)
sort edges E by weight w(u,v) in non-decreasing order
for each edge (u,v) in sorted order do
    if Find(u) ≠ Find(v) then
        A ← A ∪ {(u,v)}
        Union(u, v)
return A
```

Role of Operations:

- **`Make-Set(v)`**: Creates a singleton set containing only vertex v.
- **`Find(u)`**: Returns a representative for the set containing u. If `Find(u) == Find(v)`, then u and v are in the same component, and an edge between them creates a cycle.
- **`Union(u, v)`**: Merges the two disjoint sets containing u and v into one.

3. Applying Kruskal's Algorithm to the Given Graph

Graph Representation:

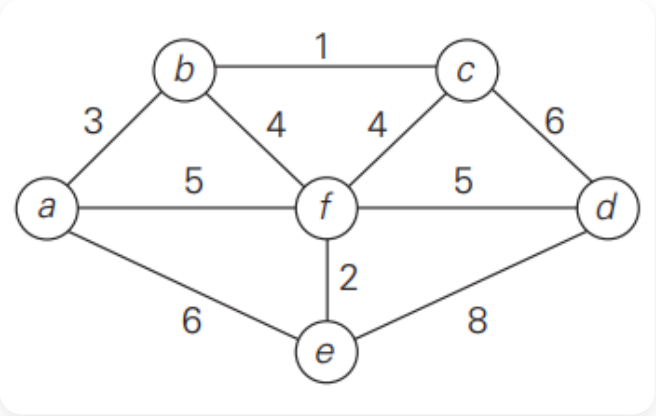


Figure: Weighted undirected graph (Vertices a,b,c,d,e,f) used in the Kruskal trace.

- **Vertices:** {a, b, c, d, e, f} ($|V| = 6$)
- **Edges with weights:** (b,c):1, (f,e):2, (a,b):3, (b,f):4, (c,f):4, (a,f):5, (f,d):5, (a,e):6, (c,d):6, (e,d):8.

Step 1 – Sort all edges by weight:

1. (b,c): 1
2. (f,e): 2
3. (a,b): 3
4. (b,f): 4
5. (c,f): 4
6. (a,f): 5
7. (f,d): 5
8. (a,e): 6
9. (c,d): 6
10. (e,d): 8

Step 2 – Initialize Union-Find:

Initial Sets: {a}, {b}, {c}, {d}, {e}, {f}

Step 3 – Iterative Edge Processing:

Iteration	Edge (u,v)	Weight	Find(u)	Find(v)	Different?	Action	Union Performed	Updated MST Edges	Running Cost
1	(b,c)	1	b	c	Yes	**Accept**	{b,c}	(b,c)	1
2	(f,e)	2	f	e	Yes	**Accept**	{f,e}	(b,c), (f,e)	3
3	(a,b)	3	a	b	Yes	**Accept**	{a,b,c}	(b,c), (f,e), (a,b)	6
4	(b,f)	4	a	f	Yes	**Accept**	{a,b,c,f,e}	(b,c), (f,e), (a,b), (b,f)	10
5	(c,f)	4	a	a	No	Reject	-	(b,c), (f,e), (a,b), (b,f)	10
6	(a,f)	5	a	a	No	Reject	-	(b,c), (f,e), (a,b), (b,f)	10
7	(f,d)	5	a	d	Yes	**Accept**	{a,b,c,f,e,d}	(b,c), (f,e), (a,b), (b,f), (f,d)	**15**

Note: Since $|V| = 6$, the MST is complete after selecting exactly $6 - 1 = 5$ edges. Remaining edges (8, 9, 10) are skipped.

Step 4 – Final MST and Total Cost:

- **MST Edges:** (b,c), (f,e), (a,b), (b,f), (f,d)
- **Total Cost:** $1 + 2 + 3 + 4 + 5 = 15$

4. Why Cycles Are Avoided and How Union-Find Detects Them

- **Cycle Avoidance:** A spanning tree must be acyclic. If we add an edge connecting two vertices that are already in the same connected component, it creates a redundant path, which is a cycle.
- **Union-Find Mechanism:** The `Find` operation determines which component a vertex belongs to. If `Find(u)` and `Find(v)` return the same representative, u and v are already connected. The algorithm rejects such edges to maintain the tree property.

5. Examination Answer Writing Format

To score maximum marks (8–10), present your answer as follows:

1. **Pseudocode (2 marks):** Write the Kruskal algorithm neatly.
2. **Sorted edge list (1 mark):** List edges in non-decreasing order.
3. **Trace table (4–5 marks):** Use a table similar to the one in Section 3 to show every step, including rejected edges.
4. **Final Result (1 mark):** State the selected edges and clearly calculate the total cost.

6. Common Mistakes

- **Skipping the Sort:** Forgetting to sort edges first; Kruskal is a greedy algorithm based on weight order.
- **Incorrect Find Logic:** Forgetting to update the component representative after a Union, which leads to adding cycles.
- **Incorrect Edge Count:** Stopping early or late; an MST must have exactly $|V| - 1$ edges.
- **Ties:** Choosing between equal weight edges incorrectly (e.g., (b,f) vs (c,f) both weight 4). Both choices should be evaluated; picking one that creates a cycle is wrong.

7. Quick Revision

- **Kruskal:** Greedy approach, sorts edges by weight, builds a forest.
- **Cycle Check:** Use Union-Find; Reject if `Find(u) == Find(v)`.
- **MST Completion:** Stop when $V - 1$ edges are added.
- **Complexity:** $O(E \log E)$.
- **For this graph:** MST Cost = **15**. Edges = **(b,c), (f,e), (a,b), (b,f), (f,d)**.

The Greedy Method: Dijkstra's Algorithm

1. What is Dijkstra's Algorithm?

- **Definition:** Dijkstra's algorithm is a **greedy algorithm** that finds the shortest paths from a single source vertex to all other vertices in a weighted graph.
- **Condition:** It works only on graphs with **non-negative edge weights**.
- **Purpose:** It is widely used in network routing protocols (like OSPF) and map applications for pathfinding.
- **Limitation:** It fails if the graph contains any negative edge weights because it assumes once a vertex is "settled," its shortest path is final.
- **Time Complexity:**
  - $O(V^2)$: Using a simple array/weight matrix.
  - $O(E \log V)$: Using an adjacency list and a priority queue (min-heap).

2. Dijkstra's Algorithm – Pseudocode

The following is the standard logical structure used for finding shortest paths:

```
 Algorithm Dijkstra(G, w, s)   // G=Graph, w=weights, s=source
for each vertex v in V do
    dist[v] ← ∞               // Initialize distances as infinite
    prev[v] ← null            // To reconstruct the path later
dist[s] ← 0                   // Distance to source is 0
Q ← all vertices              // Priority queue keyed by dist
while Q is not empty do
    u ← Extract-Min(Q)        // Pick vertex u with smallest dist
    for each neighbor v of u do
        if dist[u] + w(u,v) < dist[v] then
            dist[v] ← dist[u] + w(u,v)  // Relaxation step
            prev[v] ← u
```

Key Steps:

1. **Initialization:** Set the source distance to 0 and all others to infinity.

2. **Relaxation:** Check if going through vertex `u` provides a shorter path to neighbor `v` than currently known.
3. **Greedy Choice:** Always process the "nearest" remaining vertex from the source.

3. Applying Dijkstra's Algorithm to the Given Graph

Graph Details:

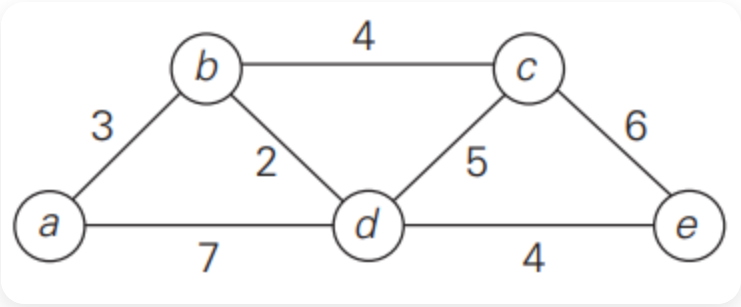


Figure: Weighted directed/undirected graph used in the trace (Vertices a,b,c,d,e). All edge weights are non-negative.

- **Vertices:** {a, b, c, d, e}
- **Edges:** (a,b):3, (a,d):7, (b,c):4, (b,d):2, (c,d):5, (c,e):6, (d,e):4

Step-by-Step Trace Table (Source = a)

Step	Extracted Vertex (u)	dist[a]	dist[b]	dist[c]	dist[d]	dist[e]	Neighbors Relaxed	Update Check ($dist[u] + w < dist[v]$)
0	-	0	∞	∞	∞	∞	-	Initial state
1	a	0	3	∞	7	∞	b, d	$0+3<\infty$ (b=3), $0+7<\infty$ (d=7)
2	b	0	3	7	5	∞	c, d	$3+4<\infty$ (c=7), $3+2<7$ (d=5)
3	d	0	3	7	5	9	c, e	$5+5>7$ (No), $5+4<\infty$ (e=9)
4	c	0	3	7	5	9	e	$7+6>9$ (No update)
5	e	0	3	7	5	9	-	All vertices processed

Final `prev` Pointers: `prev[b]=a`, `prev[d]=b`, `prev[c]=b`, `prev[e]=d`.

4. Final Shortest Paths and Distances

Destination	Shortest Distance	Shortest Path
b	3	a → b
c	7	a → b → c
d	5	a → b → d
e	9	a → b → d → e

Shortest-Path Tree Edges: {(a,b), (b,c), (b,d), (d,e)}.

5. Examination Answer Writing Format (Model Layout)

- **Algorithm/Pseudocode (2 Marks):** Write neatly and label the relaxation step.
- **Initialization (1 Mark):** Clearly show $dist[source] = 0$ and others as ∞.
- **Trace Table (4 Marks):** Show the priority queue changes and the "Relaxation Check" for every neighbor.
- **Final Result (1 Mark):** List the destination, total distance, and path sequence clearly.

6. Common Mistakes to Avoid

- **Negative Weights:** Do not use Dijkstra if there is a negative edge weight.
- **Greedy Choice:** Students often pick the "next neighbor" instead of the "overall minimum distance vertex" from the priority queue.
- **Relaxation:** Forgetting to update a distance when a shorter path is found via a different neighbor (e.g., in this graph, distance to **d** changes from 7 to 5).
- **Path Reconstruction:** Reconstructing paths forward from the source instead of backtracking from the destination using `prev` pointers.

7. Quick Revision

- **Type:** Single-Source Shortest Path (Greedy).
- **Core Logic:** $dist[v] = \min(dist[v], dist[u] + weight(u, v))$.
- **Data Structures:** Priority Queue (Min-Heap) and distance array.
- **For this graph:** The distance to **e** is **9** via path **a-b-d-e**.

THE GREEDY METHOD: Prefix Codes and Huffman Coding

1. What is a Prefix Code?

- **Definition:** A prefix code (also called a prefix-free code) is a type of variable-length code where **no codeword is a prefix of any other codeword**.
- **Simple Example:**
  - **Prefix Code:** `{0, 10, 110, 111}`. Here, `0` is not a prefix of `10`, and `110` is not a prefix of `111`.
  - **Non-Prefix Code:** `{0, 01, 10}`. This is **not** a prefix code because the codeword `0` is a prefix of `01`. This causes confusion during decoding.
- **Relation to Binary Trees:** Prefix codes are naturally represented by binary trees where every symbol is assigned to a **leaf node**. Because paths to leaves are unique and do not continue to other nodes, the prefix property is guaranteed.

2. Why Are Prefix Codes Important in Variable-Length Encoding?

Variable-length encoding is used to save space, but it can make decoding difficult. Prefix codes solve this through:

- **Unique Decodability:** Because no codeword is a prefix of another, a bit string can be parsed character by character without any ambiguity.
- **Instantaneous Decoding:** A decoder can output a symbol as soon as it recognizes a valid codeword; it does not need to "look ahead" at future bits to be sure.
- **No Need for Separators:** Fixed-length codes (like ASCII) don't need markers because we know every 8 bits is a character. Other variable-length codes might need spaces or commas, but **prefix codes require no extra bits (separators)** between codewords.
- **Optimal Compression:** Huffman codes, which are prefix codes, are mathematically proven to yield the **minimum expected bit length** for a given set of symbol frequencies.

3. Huffman Coding – Building a Prefix Code

Huffman's algorithm is a **greedy algorithm** that builds an optimal prefix tree from the bottom up.

Illustrative Example Frequencies: A:45, B:13, C:12, D:16, E:9, F:5

Step-by-Step Huffman Tree Construction:

1. **Initial State:** List all symbols as leaf nodes sorted by frequency:
   `F:5, E:9, C:12, B:13, D:16, A:45`
2. **Merge F and E:** Smallest are F(5) and E(9). Merge them into a node with frequency **14**.
   Remaining: `C:12, B:13, (14), D:16, A:45`
3. **Merge C and B:** Smallest are C(12) and B(13). Merge them into a node with frequency **25**.
   Remaining: `(14), D:16, (25), A:45`
4. **Merge (14) and D:** Smallest are (14) and D(16). Merge them into a node with frequency **30**.
   Remaining: `(25), (30), A:45`
5. **Merge (25) and (30):** Smallest are (25) and (30). Merge them into a node with frequency **55**.
   Remaining: `A:45, (55)`
6. **Final Merge:** Merge A(45) and (55) into the **Root (100)**.

Assigning Bits:

Assign **0** to every left branch and **1** to every right branch.

Final Prefix Code Table:

}

Symbol	Frequency	Codeword
A	45	0
B	13	101
C	12	100
D	16	111
E	9	1101
F	5	1100

4. Demonstrating the Prefix Property

We check the table to ensure no codeword is a prefix of another:

- **A (0)** is the only code starting with 0. All others start with 1.
- **B (101)** and **C (100)**: Neither is a prefix of the other. Both are 3 bits long and end differently.
- **D (111)**: No other code starts with `111`.
- **E (1101)** and **F (1100)**: Neither is a prefix of the other.
- **Tree Proof:** In the Huffman tree, all symbols are at the **leaves**. To reach a leaf, you must follow a unique path. Since you cannot go "through" one leaf to reach another, the prefix property is naturally satisfied.

5. Decoding with a Prefix Code (Instantaneous)

To decode, we start at the root of the tree and follow the bits until we hit a leaf.

Bit String to Decode: `1010111`

Bits Consumed	Path in Tree	Symbol Reached	Remainder
`101`	Right → Left → Right	**B**	`0111`
`0`	Left	**A**	`111`
`111`	Right → Right → Right	**D**	(Empty)

Result: The bit string `1010111` is decoded as **"BAD"**. Decoding is immediate and unambiguous.

6. Examination Answer Writing Format

For a standard **7-mark** or **10-mark** question, use this layout:

- **Definition (2 marks):** Define prefix code and mention no codeword is a prefix of another.
- **Importance (2 marks):** List unique decodability, instantaneous decoding, and no need for separators.
- **Huffman Construction (4 marks):**
  - Show sorted frequency list.
  - Draw the tree (circles for internal nodes, squares/labels for leaves).
  - Provide the final Code Table.
- **Verification (1 mark):** Briefly state that all symbols are leaves, so no prefix exists.
- **Decoding Example (1 mark):** Show a short 3-4 bit decoding trace.

7. Common Mistakes

- **Mixing up 0 and 1:** Always be consistent (e.g., Left=0, Right=1).
- **Using Internal Nodes:** Only **leaf nodes** can have symbol assignments. If you assign a symbol to an internal node, you break the prefix property.
- **Sorting Errors:** In each step, you **must** merge the two smallest values in the current list. If you merge the wrong ones, the code won't be optimal.
- **Confusing Prefix with "Unique":** All prefix codes are uniquely decodable, but not all uniquely decodable codes are prefix codes. Stick to the "no codeword is a prefix" rule.

8. Quick Revision

- **Prefix Code:** No codeword starts with another codeword.
- **Benefits:** Guarantees fast, error-free decoding without extra markers.
- **Huffman Algorithm:** Greedy approach merging the two smallest weights.
- **Tree Rule:** Symbols are always **leaves**; paths define the code.
- **Efficiency:** Huffman gives the most compact code possible for a given frequency set.

THE GREEDY METHOD: Huffman Trees and Codes

1. What is Huffman Coding?

- **Definition:** Huffman coding is a **greedy algorithm** used for constructing **optimal prefix codes**. An optimal code is one where the expected (average) codeword length is the minimum possible for a given set of symbol frequencies.
- **Purpose:** It is primarily used for **data compression**. By assigning shorter codewords to symbols that occur frequently and longer codewords to those that occur rarely, the total size of the data is reduced.
- **Greedy Choice:** At every step, the algorithm chooses the two trees (or symbols) with the **lowest frequencies** and merges them.
- **Time Complexity:** The complexity is $O(n \log n)$ when a priority queue (min-heap) is used to efficiently find the two smallest elements at each step.

2. Key Idea Behind Huffman Coding

- **Bottom-Up Construction:** The algorithm builds a binary tree from the bottom up, starting with leaf nodes for each symbol.
- **Iterative Merging:** In each iteration, the two nodes with the smallest frequencies are combined into a new internal node. The new node's frequency is the sum of the two children's frequencies.
- **Branch Labeling:** Left branches are traditionally labeled with **0** and right branches with **1** (or vice versa, provided you are consistent).
- **Prefix-Free Property:** The resulting codewords are **prefix-free**, meaning no codeword is a prefix of another. This is guaranteed because symbols are only assigned to **leaf nodes**, so a path to one symbol never passes through another.

3. Huffman Algorithm (Pseudocode)

```
Algorithm Huffman(C)
// C is the set of n characters with frequencies
```

```
n ← |C|
Q ← priority queue of nodes keyed by frequency
for each character c in C do
    create a leaf node z with z.char ← c, z.freq ← freq[c]
    Insert(z, Q)
for i ← 1 to n-1 do
    x ← Extract-Min(Q)
    y ← Extract-Min(Q)
    create a new internal node z
    z.left ← x, z.right ← y
    z.freq ← x.freq + y.freq
    Insert(z, Q)
return Extract-Min(Q)  // the root of the Huffman tree
```

Explanation:

- **Node Structure:** Each node contains a character (`char`), its frequency (`freq`), and pointers to its `left` and `right` children.
- **Priority Queue (Q):** This data structure always keeps the nodes sorted by frequency, allowing the algorithm to quickly extract the two smallest values.

4. Constructing the Huffman Tree (Step-by-Step Trace)

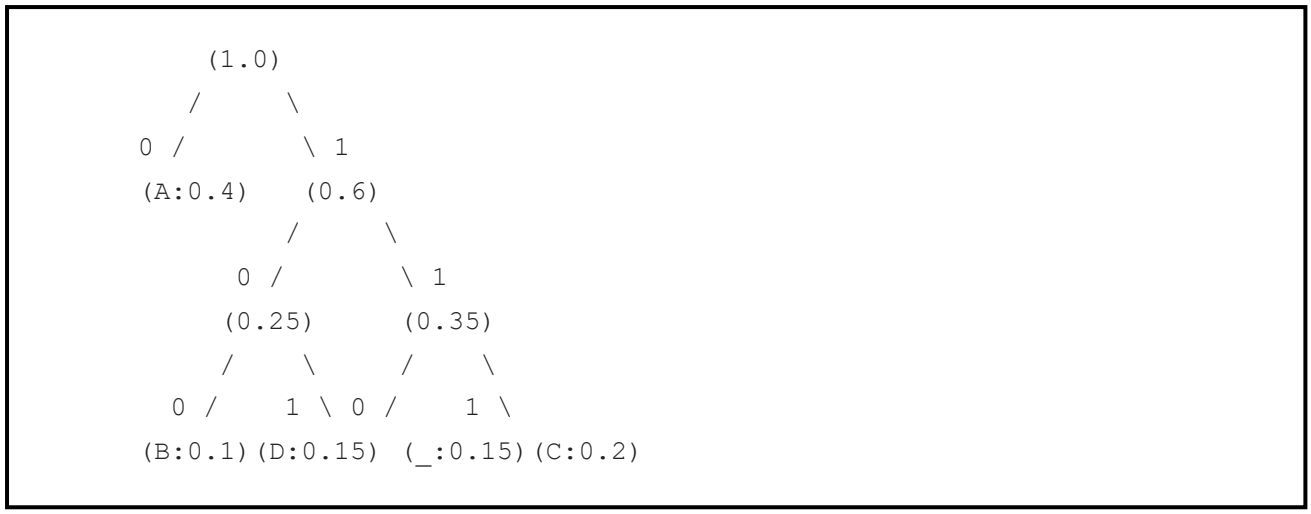
Given Data: A:0.4, B:0.1, C:0.2, D:0.15, _:0.15

Step 1 – Initial leaf nodes: Sort the symbols by frequency: B(0.1), D(0.15), _(0.15), C(0.2), A(0.4).

Step 2 – Iterative merging:
Tie-breaking rule: If frequencies are equal, pick the symbol that appeared first in the original list order.

Step	Nodes in Queue (freq)	Merge smallest two	New Node (freq)	Queue after merge
0	B:0.1, D:0.15, _:0.15, C:0.2, A:0.4	B(0.1) + D(0.15)	Node1(0.25)	_:0.15, C:0.2, Node1:0.25, A:0.4
1	_:0.15, C:0.2, Node1:0.25, A:0.4	_(0.15) + C(0.2)	Node2(0.35)	Node1:0.25, Node2:0.35, A:0.4
2	Node1:0.25, Node2:0.35, A:0.4	Node1(0.25) + Node2(0.35)	Node3(0.6)	A:0.4, Node3:0.6
3	A:0.4, Node3:0.6	A(0.4) + Node3(0.6)	**Root(1.0)**	Root(1.0)

Step 3 – Draw the final Huffman tree:



Step 4 – Extract codewords:
Based on the tree above (Left=0, Right=1):

Symbol	Frequency	Codeword
A	0.4	**0**
B	0.1	**100**
C	0.2	**111**
D	0.15	**101**
_	0.15	**110**

5. Encoding ABACABAD Using the Constructed Code

To encode, replace each symbol with its corresponding codeword from the table.

Symbol	A	B	A	C	A	B	A	D
Codeword	0	100	0	111	0	100	0	101

Final encoded string: `0100011101000101`

6. Decoding `100010111001010` Using the Constructed Code

Starting at the root, follow the bits until a leaf is reached, then reset to the root.

Step	Remaining Bits	Current Bit	Move in Tree	Output Symbol
1	**100**010111001010	1, 0, 0	Root → Right → Left → Left	**B**
2	010111001010	0	Root → Left	**A**
3	**101**11001010	1, 0, 1	Root → Right → Left → Right	**D**
4	**110**01010	1, 1, 0	Root → Right → Right → Left	_
5	01010	0	Root → Left	**A**
6	**101**0	1, 0, 1	Root → Right → Left → Right	**D**
7	0	0	Root → Left	**A**

Final decoded string: `BAD_ADA`

7. Final Results Summary

- **Codeword Table:** A:0, B:100, C:111, D:101, _:110.
- **Encoded** `ABACABAD`**:** `0100011101000101`.
- **Decoded** `100010111001010`**:** `BAD_ADA`.

8. Examination Answer Writing Format (For 10 Marks)

1. **Huffman Tree Construction (4 Marks):** Show the initial leaf nodes and the iterative merge steps (table). Draw the final tree clearly with frequency labels and 0/1 edges.
2. **Codeword Table (1 Mark):** Present the final symbol-to-bits mapping.
3. **Encoding (2 Marks):** Show the symbol-by-symbol substitution for `ABACABAD` and the final string.
4. **Decoding (3 Marks):** Show the stepwise bit processing for `100010111001010` and state the final result.

9. Common Mistakes to Avoid

- **Inconsistent Labeling:** Swapping the 0/1 rule halfway through the tree.
- **Non-Leaf Symbols:** Attempting to assign codewords to internal nodes (only leaves represent symbols).
- **Merging Errors:** Failing to sort the queue after a merge, which leads to merging the wrong "smallest" nodes.
- **Decoding Reset:** Forgetting to return to the root of the tree after finding a symbol.

10. Quick Revision Box

- **Strategy:** Greedy (merge two smallest).
- **Complexity:** $O(n \log n)$.
- **Property:** Prefix-free (instant decoding).
- **For this data:** A is the shortest (1 bit) because it is the most frequent (0.4).
- **Result:** `ABACABAD` → `0100011101000101`.
- **Result:** `100010111001010` → `BAD_ADA`.

THE GREEDY METHOD: Huffman Trees and Codes

1. Huffman Tree Construction (Step-by-Step Trace)

Huffman's algorithm is a greedy approach that builds an optimal prefix tree from the bottom up by repeatedly merging the two nodes with the smallest frequencies.

Initial Sorted List of Nodes:
B (0.1), _ (0.15), C (0.2), D (0.2), A (0.35).

Merging Step Table:

Step	Queue (Sorted Frequencies)	Merge	New Node (Combined Freq)	Queue After Merge
1	B:0.1, _:0.15, C:0.2, D:0.2, A:0.35	B + _	N1 (0.25)	C:0.2, D:0.2, N1:0.25, A:0.35
2	C:0.2, D:0.2, N1:0.25, A:0.35	C + D	N2 (0.40)	N1:0.25, A:0.35, N2:0.40
3	N1:0.25, A:0.35, N2:0.40	N1 + A	N3 (0.60)	N2:0.40, N3:0.60
4	N2:0.40, N3:0.60	N2 + N3	**Root (1.0)**	(Empty)

Tie-breaking rule: At each step, the two smallest weights are merged. When frequencies are equal (as with C and D at 0.2), they are merged into a single internal node before merging with larger remaining values.

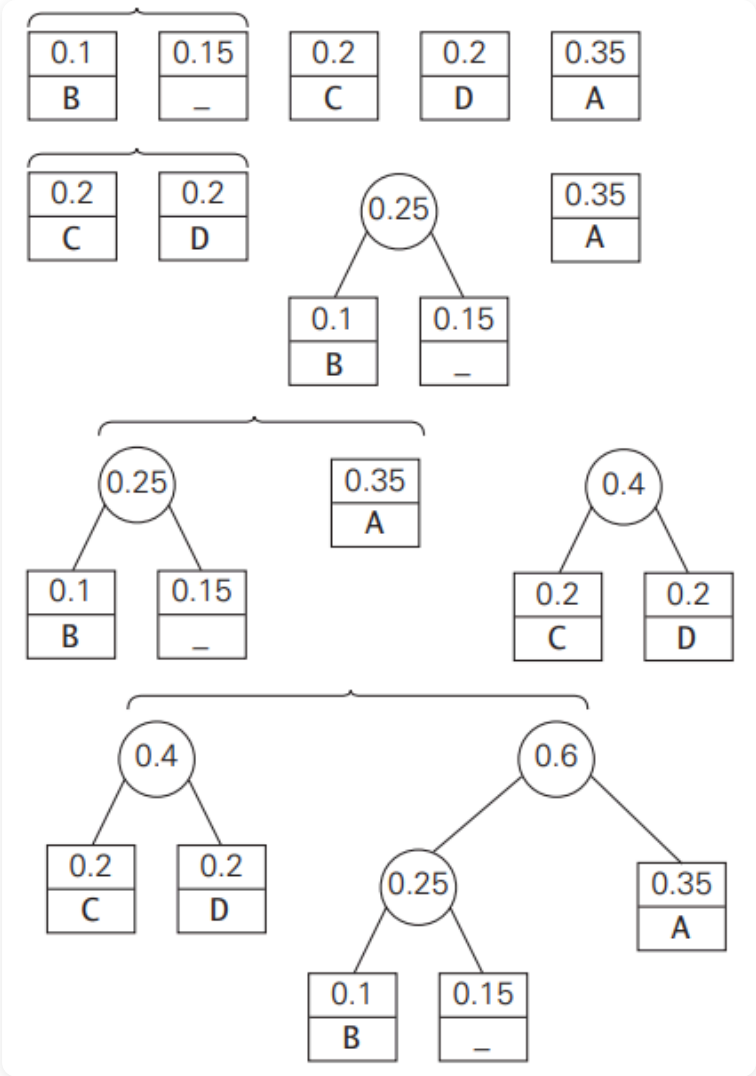


Figure: Final Huffman tree (left=0, right=1) showing internal node frequencies and leaf symbols.

Final Huffman Tree (ASCII representation):

Assign **0** to left branches and **1** to right branches.

```
                [ 1.0 ] (Root)
               /       \
          0 /           \ 1
        [ 0.4 ]         [ 0.6 ]
       /       \        /      \
    0 /      1 \    0 /        \ 1
  C(0.2)    D(0.2)  [0.25]     A(0.35)
                    /    \
                 0 /      \ 1
               B(0.1)     _(0.15)
```

2. Resulting Codewords

The codeword for a symbol is obtained by recording the labels on the path from the root to that symbol's leaf.

Symbol	Frequency	Codeword
A	0.35	11
B	0.1	100
C	0.2	00
D	0.2	01
_	0.15	101

Prefix-free Confirmation: No codeword is a prefix of another (e.g., "00" does not start any other code); this is guaranteed because symbols are assigned only to leaf nodes.

3. Encoding DAD

To encode the string, each symbol is replaced by its corresponding Huffman codeword.

Symbol	Codeword
D	01
A	11
D	01

Final Encoded Bit String: `011101`.

4. Decoding `10011011011101`

Decoding is performed by scanning the bit string and following the tree paths until a leaf is reached, then returning to the root for the next bits.

Step	Remaining Bits	Next Bit(s)	Node Path	Output Symbol	Remaining After
1	10011011011101	1, 0, 0	root → 1 → 0 → 0	**B**	11011011101
2	11011011101	1, 1	root → 1 → 1	**A**	011011101
3	011011101	0, 1	root → 0 → 1	**D**	1011101
4	1011101	1, 0, 1	root → 1 → 0 → 1	_	1101
5	1101	1, 1	root → 1 → 1	**A**	01
6	01	0, 1	root → 0 → 1	**D**	(empty)

Final Decoded String: `BAD_AD`.

5. Compression Ratio

- **Average Huffman Bits per Symbol:**
  $\sum(\text{Frequency} \times \text{Code Length}) = (0.35 \times 2) + (0.1 \times 3) + (0.2 \times 2) + (0.2 \times 2) + (0.15 \times 3)$

  $= 0.70 + 0.30 + 0.40 + 0.40 + 0.45 = \mathbf{2.25\ bits/symbol}$.
- **Fixed-Length Baseline:**
  For an alphabet of 5 symbols, fixed-length encoding requires $m \geq \log_2 5$ bits, which is **3 bits/symbol**.
- **Compression Ratio Definition:**
  The effectiveness is measured by the percentage of memory saved compared to the baseline.
- **Calculation:**
  $\text{Savings} = \frac{\text{Fixed Bits} - \text{Huffman Average Bits}}{\text{Fixed Bits}} \times 100\%$
  $= \frac{3 - 2.25}{3} \times 100\% = \frac{0.75}{3} \times 100\% = \mathbf{25\%}$.

> ✅ Huffman coding achieves a **25%** reduction in space for this text compared to a fixed-length code.
